

Szkolenie: Programowanie Python na poziomie podstawowym. Akademia IT

Trener: Radosław Janiak

Geneza Pythona

1989–1991

Guido van Rossum rozpoczyna pracę nad Pythonem. Pierwsze publiczne wydanie ukazuje się w 1991 roku.

Nazwa z Monty Pythona

Nazwa języka nawiązuje do grupy komediowej Monty Python.

Python 3

Pierwsze wydanie: 2008. Wsparcie Pythona 2 zakończyło się 1 stycznia 2020 roku.

Rozwój języka

Propozycje zmian opisują dokumenty PEP. Na szkoleniu używamy Pythona 3.10 lub nowszego.

Cechy Pythona

Zastosowanie i styl

Język ogólnego przeznaczenia, nastawiony na czytelność. Łączy styl strukturalny, obiektowy i elementy funkcyjne.

Implementacje

CPython powstaje głównie w C. Inne implementacje to m.in. PyPy, Jython i IronPython. CLPython jest przykładem historycznym.

Typowanie i pamięć

Typy obiektów sprawdzane są dynamicznie. Pamięcią zarządza interpreter. CPython używa zliczania referencji i garbage collectora cykli.

Skrypty i aplikacje

Python bywa nazywany językiem skryptowym. Nadaje się też do budowania większych aplikacji i usług.

Przekazywanie argumentów

Parametr wiąże się z przekazanym obiektem. Zmiana listy może być widoczna poza funkcją, ale lokalne przypisanie nie podmienia zmiennej wywołującego.

Enkapsulacja

Klasy łączą dane z metodami. _nazwa oznacza szczegół wewnętrzny, a __nazwa ogranicza kolizje. Nie ma wymuszania dostępu jak private w Javie.

Zmienna liczba argumentów

Funkcja może zbierać dodatkowe argumenty przez *args i **kwargs. Otrzymuje je jako krotkę i słownik.

Wartości domyślne

Parametr może mieć wartość używaną, gdy argument pominięto. Pozwala to uprościć typowe wywołania funkcji.

Python w analizie danych

Python jest jednym z narzędzi Data Science. R ma silne korzenie w statystyce. Wybór zależy od zadania, bibliotek i zespołu.

Od analizy do aplikacji

Język ogólnego przeznaczenia pozwala połączyć analizę danych z aplikacją desktopową lub webową. Wyniki obliczeń mogą stać się częścią większego rozwiązania.

Szybkie eksperymentowanie

Interpreter i notatniki Jupyter pozwalają uruchamiać małe fragmenty kodu oraz od razu oglądać wyniki. Ułatwia to prototypowanie i sprawdzanie kolejnych pomysłów.

Biblioteki obliczeniowe i AI

NumPy, pandas i SciPy wspierają obliczenia oraz analizę danych. Matplotlib służy do wizualizacji, a scikit-learn udostępnia narzędzia uczenia maszynowego.

Społeczność i dokumentacja

Dokumentacja, przykłady i dyskusje pomagają rozwiązywać problemy. Rozbudowany ekosystem bibliotek ogranicza potrzebę pisania każdej funkcji od początku.

Cechy — zastosowania Pythona

Graphical User Interface

Qt GTK Tkinter

Okna, formularze i aplikacje desktopowe

Web Development

django Flask FastAPI

Strony internetowe i API

System Administration

THE LINUX FOUNDATION LSB

os, pathlib, subprocess — skrypty systemowe



Productivity

PyUNO The Document Foundation

openpyxl, XlsxWriter — praca z Excelem

Data Science

pandas matplotlib

NumPy, SciPy, Polars, Seaborn

Machine Learning

PyTorch TensorFlow

scikit-learn, Keras

Web Scraping

Requests, Beautiful Soup
Scrapy, Selenium

Automation Testing

pytest, unittest
Robot Framework

Image Processing

OpenCV, scikit-image
Pillow

Game Development

Pygame, Arcade
Panda3D

Dynamiczne typowanie i adnotacje

Typ należy do obiektu, a nazwa może wskazać później inny obiekt. Adnotacja opisuje oczekiwany typ dla czytelnika i narzędzi, ale sama nie wymusza go podczas działania programu.

PRZYKŁAD

```
wartosc = 25
print(type(wartosc).__name__)
wartosc = "dwadzieścia pięć"
print(type(wartosc).__name__)

wiek: int = 25
wiek = "25"  # celowo niezgodne z adnotacją
print(type(wiek).__name__)
```

WYNIK

```
int
str
str
```

Edytor lub narzędzie do analizy typów może zgłosić niezgodność.

Konwersję wykonuje np. int("25").

Co otrzymasz po wykonaniu int("abc")?

Operatory arytmetyczne

Najpierw wykonujemy potęgowanie, potem mnożenie i dzielenie, a na końcu dodawanie i odejmowanie. Nawiasy zmieniają tę kolejność. Dla napisów + łączy teksty, a * powtarza tekst.

PRZYKŁAD

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(2 ** 3)
print("Py" + "thon")
print("ha" * 3)
```

WYNIK

```
14
20
3.5
3
1
8
Python
hahaha
```

/ dzielenie
// dzielenie z zaokrągleniem w dół
% reszta
** potęgowanie

Jaki wynik da -7 // 2?

Porównania i wartości logiczne

Porównania pozwalają sprawdzić relacje między wartościami i zwracają True lub False. Warunki można łączyć przez and i or oraz odwracać przez not. bool() sprawdza prawdziwość wartości.

PRZYKŁAD

```
wiek = 20
print(wiek >= 18)
print(18 <= wiek < 65)
print(wiek < 18 or wiek >= 65)
print(not (wiek == 20))
print(bool(""))
print(bool([]))
print(bool("False"))
```

WYNIK

```
True
True
False
False
False
False
True
```

Fałszywe są m.in. None, False, zera oraz puste napisy i kolekcje.

Dlaczego napis "False" jest prawdziwy w warunku?

Liczby i konwersja typów

int przechowuje liczby całkowite, float liczby zmiennoprzecinkowe, a complex liczby zespolone. Konwersja tworzy wartość innego typu, np. zamienia napis z cyframi na liczbę do obliczeń.

PRZYKŁAD

```
liczba = int("123")
print(liczba + 1)
print(float(5))
print(int(5.9))
print(0.1 + 0.2)
print(f"{0.1 + 0.2:.2f}")
```

WYNIK

```
124
5.0
5
0.30000000000000004
0.30
```

float ma ograniczoną precyzję.

Format .2f zmienia sposób wyświetlania, a nie wartość w pamięci.

Co zwróci int(-5.9)?

Napisy: indeksy i wycinki

Napis str jest niemutowalną sekwencją znaków Unicode. Indeksy liczymy od zera, a ujemne indeksy od końca. Wycinek pozwala pobrać fragment tekstu bez zmiany oryginału.

PRZYKŁAD

```
imie = "Tomasz"
print(imie[0])
print(imie[-1])
print(imie[1:4])
print(imie.lower())
print(imie)
print(len(imie))
```

WYNIK

```
T
z
oma
tomasz
Tomasz
6
```

Wycinek [1:4] obejmuje indeksy 1, 2 i 3.

lower() zwraca nowy napis.

Czy przypisanie imie[0] = "R" jest dozwolone?

Formatowanie napisów

f-string łączy tekst z wartościami zmiennych i wynikami obliczeń. Wyrażenia w nawiasach klamrowych są obliczane przy tworzeniu napisu, a zapis po dwukropku steruje formatem wyniku.

PRZYKŁAD

```
imie = "Ala"
wiek = 25
wynik = 7 / 3
print(f"{imie} ma {wiek} lat.")
print(f"Za rok: {wiek + 1}")
print(f"Wynik: {wynik:.2f}")
```

WYNIK

```
Ala ma 25 lat.
Za rok: 26
Wynik: 2.33
```

Prefiks f włącza formatowanie.

Wyrażenia zapisujemy wewnątrz { }.

Zmień precyzję wyniku na trzy miejsca po przecinku.

Listy

Lista przechowuje elementy w określonej kolejności i pozwala je dodawać, usuwać oraz podmieniać. Może zawierać różne typy, także inne listy. Element wybieramy przez jego indeks.

PRZYKŁAD

```
liczby = [1, 2, 3]
liczby.append(4)
liczby[0] = 10
print(liczby)
nowa = liczby + [5]
print(nowa)
print(liczby)
```

WYNIK

```
[10, 2, 3, 4]
[10, 2, 3, 4, 5]
[10, 2, 3, 4]
```

append() zmienia listę.

Operator + tworzy nową listę.

extend() dodaje wiele elementów do istniejącej listy.

Co zmieni liczby.extend([5, 6])?

Słowniki

Słownik dict wiąże klucze z wartościami, np. nazwę pola z danymi osoby. Dostęp odbywa się przez klucz. Słownik zachowuje kolejność dodawania kluczy, a ponowny zapis pod kluczem zmienia jego wartość.

PRZYKŁAD

```
osoba = {"imie": "Ala", "wiek": 25}
print(osoba["imie"])
osoba["miasto"] = "Kraków"
print(list(osoba))
print("wiek" in osoba)
print(osoba.get("telefon", "brak"))
```

WYNIK

```
Ala
['imie', 'wiek',
'miasto']
True
brak
```

Klucze są unikalne i muszą być hashowalne, np. str lub int.

get() pozwala podać wartość domyślną.

Co zwróci osoba["telefon"]?

Krotki i rozpakowywanie

Krotka tuple grupuje elementy w ustalonej kolejności, np. współrzędne punktu. Po utworzeniu nie można podmienić jej elementów. Rozpakowanie pozwala przypisać je do oddzielnych nazw.

PRZYKŁAD

```
punkt = (10, 20)
x, y = punkt
print(x)
print(y)
jedna = (5,)
print(type(jedna).__name__)
print(list(punkt))
```

WYNIK

```
10
20
tuple
[10, 20]
```

Przecinek tworzy krotkę jednoelementową.

Rozpakowanie przypisuje elementy do osobnych nazw.

Czym różni się (5) od (5,)?

Zbiory

Zbiór set przechowuje unikalne elementy i usuwa powtórzenia. Przydaje się do sprawdzania przynależności i porównywania kolekcji. Kolejność wyniku pokazujemy tu przez sorted(), które zwraca listę.

PRZYKŁAD

```
a = {1, 2, 2, 3}
b = {3, 4}
print(sorted(a))
print(sorted(a | b))
print(sorted(a & b))
print(sorted(a - b))
print(sorted(a ^ b))
```

WYNIK

```
[1, 2, 3]
[1, 2, 3, 4]
[3]
[1, 2]
[1, 2, 4]
```

| suma
& część wspólna
- różnica
^ różnica symetryczna

Pusty zbiór: set().

Jaki typ tworzy zapis {}?

Sprawdź, co już pamiętasz

3 minuty z fiszkami

Otwórz naukapythona.pl lub zeskanuj kod QR.

Sprawdź poznane pojęcia.

Wybierz jedno, które chcesz jeszcze omówić.



naukapythona.pl

Instrukcje if, elif i else

Python sprawdza warunki od góry i wykonuje pierwszą pasującą gałąź. Kolejne elif są wtedy pomijane. Blok else wykona się, jeśli żaden wcześniejszy warunek nie był prawdziwy.

PRZYKŁAD

```
liczba = 42
if liczba < 10:
    print("Mniej niż 10")
elif liczba < 100:
    print("Od 10 do 99")
else:
    print("Co najmniej 100")
```

WYNIK

```
Od 10 do 99
```

Wcięcie wyznacza blok kodu.

else obsługuje pozostałe przypadki.

Jaką gałąź wybierze program dla liczby 100?

None i pusty napis

None oznacza brak wartości, a pusty napis jest istniejącą wartością typu str. Obie wartości są fałszywe w warunku, ale nie są sobie równe. Rozróżnienie pomaga np. odróżnić brak danych od pustego pola.

PRZYKŁAD

```
wynik = None
tekst = ""
print(wynik is None)
print(tekst == "")
print(bool(wynik))
print(bool(tekst))
print(wynik == tekst)
```

WYNIK

```
True
True
False
False
False
```

Brak wartości sprawdzamy przez is None.

Dwie wartości fałszywe w warunku nie muszą być sobie równe.

Czy 0 == False oraz 0 is False oznaczają to samo?

Uruchomienie pliku i import

Podczas uruchomienia pliku Python nadaje zmiennej __name__ wartość "__main__". Przy imporcie jest to nazwa modułu. Warunek pozwala uruchamiać część programu tylko przy bezpośrednim starcie pliku.

PRZYKŁAD

```
# plik powitanie.py
print("Ładowanie modułu")

if __name__ == "__main__":
    print("Uruchomiono plik bezpośrednio")
```

WYNIK

```
Ładowanie modułu
Uruchomiono plik
bezpośrednio
```

Przy imporcie wykona się pierwszy print.

Blok if uruchomi się przy bezpośrednim starcie pliku.

Co zobaczysz po import powitanie?

Dopasowywanie wzorców: match/case

Instrukcja match/case, dostępna od Pythona 3.10, wybiera działanie na podstawie dopasowanego wzorca. Tutaj porównujemy kod z konkretnymi liczbami. Wykona się pierwsza pasująca gałąź.

PRZYKŁAD

```
kod = 404
match kod:
    case 200:
        print("OK")
    case 404:
        print("Nie znaleziono")
    case _:
        print("Inny kod")
```

WYNIK

```
Nie znaleziono
```

_ dopasowuje pozostałe przypadki.

match potrafi dopasować także strukturę danych, np. listy.

Która gałąź obsłuży wartość 500?

Ćwiczenie: warunki

Napisz program opisujący temperaturę: poniżej 0 „mróz”, od 0 do 19 „chłodno”, a od 20 „ciepło”. Zacznij od wartości 18 i użyj if/elif/else. Przy każdym uruchomieniu wypisz jeden komunikat.

KOD DO UZUPEŁNIENIA

```
temperatura = 18
# Dopisz warunki.
```

OCZEKIWANY WYNIK

```
Dla 18: chłodno
Dla -2: mróz
Dla 20: ciepło
```

Sprawdź także 0, 19 i 20. Każda z tych wartości powinna trafić dokładnie do jednej gałęzi.

Czas: 3 minuty

Pętla for

Pętla for wykonuje blok dla kolejnych elementów, np. imion na liście. enumerate() udostępnia jednocześnie numer i element. Gdy potrzebujesz ciągu liczb, możesz użyć range().

PRZYKŁAD

```
imiona = ["Ala", "Jan", "Ewa"]
for numer, imie in enumerate(imiona, start=1):
    print(numer, imie)

for liczba in range(3):
    print(liczba)
```

WYNIK

```
1 Ala
2 Jan
3 Ewa
0
1
2
```

range(3) daje 0, 1, 2.

Po słowniku można przejść przez items(), otrzymując klucz i wartość.

Jak otrzymać liczby 1, 2 i 3 przez range()?

Pętla while

Pętla while sprawdza warunek przed każdym przebiegiem i działa, dopóki jest on prawdziwy. Kod w pętli powinien zbliżać program do zakończenia. Tutaj zwiększamy licznik w każdym przebiegu.

PRZYKŁAD

```
licznik = 0
while licznik < 5:
    licznik += 1
    if licznik == 2:
        continue
    if licznik == 4:
        break
    print(licznik)
```

WYNIK

```
1
3
```

continue pomija resztę bieżącego przebiegu.

break kończy pętlę.

Co wypisze program po usunięciu obu instrukcji if?

List comprehensions

Wyrażenie listowe tworzy nową listę przez przekształcenie kolejnych elementów. Opcjonalny warunek if pozwala wybrać tylko część danych. Kolejność wyniku odpowiada kolejności przetwarzania.

PRZYKŁAD

```
liczby = [1, 2, 3, 4, 5]
kwadraty = [x ** 2 for x in liczby]
parzyste = [x for x in liczby if x % 2 == 0]
print(kwadraty)
print(parzyste)
```

WYNIK

```
[1, 4, 9, 16, 25]
[2, 4]
```

Najpierw zapisz czytelną pętlę. Potem rozważ krótszą formę.

Istnieją też wyrażenia dla zbiorów i słowników.

Jak otrzymać kwadraty tylko liczb parzystych?

Ćwiczenie: pętle

Dla listy [4, 12, 7, 18, 10, 21] policz elementy większe od 10. Użyj pętli for, warunku if i licznika. Po zakończeniu pętli wypisz liczbę pasujących elementów, a nie ich sumę.

KOD DO UZUPEŁNIENIA

```
liczby = [4, 12, 7, 18, 10, 21]
# Policz liczby większe od 10.
```

OCZEKIWANY WYNIK

```
3
```

Ustaw licznik na 0 przed pętlą. Zwiększaj go tylko przy spełnionym warunku. Liczba 10 nie powinna zostać policzona.

Czas: 4 minuty

PEP 8: wcięcia i długie linie

Wcięcia określają przynależność instrukcji do bloku, dlatego wpływają na działanie programu. PEP 8 zaleca 4 spacje na poziom. Długie wyrażenia można rozłożyć na linie wewnątrz nawiasów.

PRZYKŁAD

```
punkty = [
    10,
    20,
    30,
]
if sum(punkty) >= 50:
    print("Cel osiągnięty")
```

WYNIK

```
Cel osiągnięty
```

PEP 8 zaleca do 79 znaków kodu w linii.

Zasady projektu mogą ustalać inny limit.

Które linie należą do bloku if?

PEP 8: nazwy i odstępy

Nazwy powinny mówić, co przechowuje zmienna lub co robi funkcja. Spójne odstępy i puste linie ułatwiają rozpoznanie struktury programu. PEP 8 opisuje konwencje, które pomagają czytać cudzy kod.

PRZYKŁAD

```
def pole_prostokata(a, b):
    return a * b


def obwod_prostokata(a, b):
    return 2 * (a + b)


print(pole_prostokata(3, 4))
```

WYNIK

```
12
```

Funkcje i zmienne: snake_case.
Klasy: PascalCase.

Dwie puste linie między definicjami na poziomie modułu.

Jak nazwiesz funkcję przeliczającą kilometry na metry?

Importowanie modułów

Import udostępnia funkcje, klasy i inne nazwy z modułu. Zapis import math pozwala używać math.sqrt(), a from math import sqrt udostępnia samo sqrt(). Wybierz formę czytelną w danym programie.

PRZYKŁAD

```
import math
from pathlib import Path

print(math.sqrt(81))
print(Path("dane.csv").suffix)
```

WYNIK

```
9.0
.csv
```

Kolejność grup importów:
1. Biblioteka standardowa.
2. Pakiety zewnętrzne.
3. Moduły projektu.

Czym różni się import math od from math import sqrt?

Własny moduł i pakiet

Moduł to zwykle plik .py, którego kod można wykorzystać w innym pliku przez import. Zwykły pakiet grupuje moduły w katalogu z __init__.py. Taki podział ułatwia porządkowanie i ponowne użycie kodu.

PRZYKŁAD

```
# plik matma.py

def dodaj(a, b):
    return a + b
```

WYNIK

```
3
```

W drugim pliku, w tym samym katalogu:

```
import matma
print(matma.dodaj(1, 2))
```

Dodaj do modułu funkcję odejmij(a, b).

Środowisko wirtualne i pip

Środowisko wirtualne oddziela pakiety projektu od pakietów innych projektów. venv tworzy środowisko, a pip instaluje w nim biblioteki. Dzięki temu projekty mogą używać różnych wersji zależności.

POLECENIA W TERMINALU

```
python -m venv .venv

# Windows PowerShell
.\.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate

# Po aktywacji
python -m pip install requests
python -m pip list
```

Wybierz polecenie aktywacji dla swojego systemu.

Na macOS / Linux tworzenie środowiska może wymagać python3.

Wyjście: deactivate.

W jakim środowisku instaluje pakiet python -m pip?

Definiowanie funkcji

Funkcja zamyka fragment działania pod nazwą, aby można było go wielokrotnie wywołać. Argumenty dostarczają dane, a return przekazuje wynik. Wartość domyślna pozwala pominąć wybrany argument.

PRZYKŁAD

```
def powiel(tekst, ile_razy=2):
    return tekst * ile_razy

print(powiel("ha"))
print(powiel("ha", ile_razy=3))
```

WYNIK

```
haha
hahaha
```

ile_razy ma wartość domyślną.

return zwraca wartość. print() wyświetla tekst.

Brak return oznacza wynik None.

Co zwróci powiel("ha", 0)?

Argumenty *args i **kwargs

Zapis *args zbiera dodatkowe argumenty pozycyjne do krotki, a **kwargs argumenty nazwane do słownika. Dzięki temu funkcja może przyjmować zmienną liczbę danych. Znaczenie mają gwiazdki, nie same nazwy.

PRZYKŁAD

```
def pokaz(*args, **kwargs):
    print(args)
    print(kwargs)

pokaz(3, 4, imie="Ala")
```

WYNIK

```
(3, 4)
{'imie': 'Ala'}
```

args jest krotką.
kwargs jest słownikiem.

Parametry i lokalne przypisania należą do wywołania funkcji.

Co zmieni pokaz(3, 4, 5, miasto="Łódź")?

Ćwiczenie: funkcje

Uzupełnij funkcję czy_parzysta(liczba), aby zwracała True dla liczb parzystych i False dla nieparzystych. Wynik ma być wartością bool zwróconą przez return. Wyświetl go dopiero przy wywołaniu funkcji.

KOD DO UZUPEŁNIENIA

```
def czy_parzysta(liczba):
    pass

print(czy_parzysta(8))
print(czy_parzysta(7))
```

OCZEKIWANY WYNIK

```
True
False
```

Użyj % do sprawdzenia reszty z dzielenia przez 2. Przetestuj także 0 i liczbę ujemną. Zastąp instrukcję pass własnym kodem.

Czas: 4 minuty

Lambda, map i filter

lambda tworzy małą funkcję, której wynikiem jest wartość jednego wyrażenia. Można przekazać ją do map() lub filter(). list() zbiera wyniki iteratora, aby dało się je wyświetlić jako listę.

PRZYKŁAD

```
liczby = [1, 3, 5, 7]
podwojone = map(lambda x: x * 2, liczby)
wieksze = filter(lambda x: x > 3, liczby)
print(list(podwojone))
print(list(wieksze))
```

WYNIK

```
[2, 6, 10, 14]
[5, 7]
```

map() przekształca elementy.
filter() wybiera elementy.

W Pythonie 3 oba zwracają iteratory.

Zapisz ten sam wybór za pomocą list comprehension.

Czytelna funkcja zamiast zagnieżdżeń

Gdy logika obejmuje kilka przypadków, nazwane funkcje i osobne warunki ułatwiają jej zrozumienie. Każde return kończy bieżące wywołanie. Dlatego kolejny warunek dotyczy już pozostałych przypadków.

PRZYKŁAD

```
def grupa_wiekowa(wiek):
    if wiek < 10:
        return "dziecko"
    if wiek < 18:
        return "nastolatek"
    return "dorosły"

print(grupa_wiekowa(18))
```

WYNIK

```
dorosły
```

lambda pasuje do krótkiego wyrażenia.

Wielogałęziową logikę można jasno zapisać przez def.

Jak sprawdzisz granice: 9, 10, 17 i 18?

Klasa, obiekt i metoda

Klasa opisuje dane i zachowania obiektów, a każde jej wywołanie może utworzyć osobną instancję. Atrybuty przechowują stan obiektu. Metody korzystają z tego stanu przez parametr self.

PRZYKŁAD

```
class Pojazd:
    def __init__(self, marka):
        self.marka = marka

    def jedz(self):
        return f"{self.marka} jedzie"

pojazd = Pojazd("Ford")
print(pojazd.jedz())
```

WYNIK

```
Ford jedzie
```

__init__ inicjalizuje obiekt.
self wskazuje bieżący obiekt.
self.marka jest jego atrybutem.

Utwórz drugi pojazd z inną marką.

Dziedziczenie i nadpisanie metody

Dziedziczenie pozwala klasie pochodnej korzystać z atrybutów i metod klasy bazowej. Może ona także zdefiniować własną wersję metody. Wywołanie na obiekcie Rower wybierze wtedy wersję z tej klasy.

PRZYKŁAD

```
class Pojazd:
    sygnal = "bip"
    def hamuj(self):
        return "hamuję"

class Rower(Pojazd):
    def hamuj(self):
        return "hamuję rowerem"

print(Rower().hamuj())
print(Rower().sygnal)
```

WYNIK

```
hamuję rowerem
bip
```

Rower dziedziczy sygnal.

Metoda hamuj() w Rower zastępuje zachowanie z Pojazd.

Co zwróci Pojazd().hamuj()?

Atrybuty i enkapsulacja

self.atrybut

Przechowuje dane konkretnego obiektu. Atrybut zdefiniowany w ciele klasy należy do klasy i może być wspólny dla jej instancji.

_atrybut

Oznacza szczegół wewnętrzny, z którego kod zewnętrzny nie powinien korzystać. To konwencja współpracy, która sama nie blokuje odczytu ani zmiany.

__atrybut

Python przekształca nazwę, aby ograniczyć przypadkowe kolizje przy dziedziczeniu. Nadal można uzyskać dostęp do atrybutu, więc mechanizm nie stanowi zabezpieczenia.

Metody i właściwości

Enkapsulacja udostępnia publiczny interfejs do pracy ze stanem obiektu. Metoda lub @property może sprawdzić poprawność wartości przed jej zapisaniem.

Pliki tekstowe i with

open() otwiera plik w wybranym trybie i kodowaniu. Blok with zapewnia jego zamknięcie także po wyjątku. Najpierw zapisujemy tekst, a następnie otwieramy plik ponownie, aby odczytać jego zawartość.

PRZYKŁAD

```
with open("notatka.txt", "w", encoding="utf-8") as plik:
    plik.write("Cześć, Python!")

with open("notatka.txt", encoding="utf-8") as plik:
    tekst = plik.read()

print(tekst)
```

WYNIK

```
Cześć, Python!
```

r: odczyt
w: zapis od początku
a: dopisywanie

Ścieżka względna odnosi się do katalogu roboczego.

Co stanie się ze starym plikiem przy trybie "w"?

CSV: odczyt

CSV przechowuje dane tabelaryczne w wierszach, z polami oddzielonymi separatorem. DictReader zamienia każdy wiersz na słownik. Plik osoby.csv ma nagłówki imie,wiek i jeden wiersz danych: Ala,25.

PRZYKŁAD

```
import csv

with open("osoby.csv", newline="", encoding="utf-8") as plik:
    czytnik = csv.DictReader(plik)
    for osoba in czytnik:
        print(osoba["imie"], int(osoba["wiek"]))
```

WYNIK

```
Ala 25
```

DictReader używa pierwszego wiersza jako nazw kolumn.

Wartości z CSV są napisami.

Co trzeba zmienić, jeśli separatorem jest średnik?

CSV: zapis

csv.writer zapisuje pola z uwzględnieniem separatorów i cudzysłowów. writerow() przyjmuje jedną sekwencję wartości, np. listę. Najpierw zapisujemy nazwy kolumn, a potem wiersz danych osoby.

PRZYKŁAD

```
import csv

with open("osoby.csv", "w", newline="",
          encoding="utf-8") as plik:
    zapis = csv.writer(plik)
    zapis.writerow(["imie", "wiek"])
    zapis.writerow(["Ala", 25])
```

ZAWARTOŚĆ PLIKU

```
imie,wiek
Ala,25
```

newline="" pozwala modułowi csv obsługiwać zakończenia wierszy.

writerows() zapisuje wiele wierszy.

Dopisz drugi wiersz z imieniem i wiekiem.

Ćwiczenie: pliki

Zapisz miasta z podanej listy do pliku miasta.txt, każde w osobnej linii. Następnie otwórz plik do odczytu i wyświetl jego zawartość. Zachowaj kolejność miast oraz polskie znaki.

KOD DO UZUPEŁNIENIA

```
miasta = ["Łódź", "Gdańsk", "Opole"]
# Zapisz i odczytaj plik.
```

OCZEKIWANY WYNIK

```
Łódź
Gdańsk
Opole
```

Użyj with i kodowania utf-8. Między miastami wstaw znak nowej linii. Porównaj odczytany tekst z wynikiem po prawej.

Czas: 5 minut

Docstring i dokumentacja

Docstring to napis na początku funkcji, klasy lub modułu, który opisuje ich przeznaczenie. Python przechowuje go w __doc__, a help() używa go do wyświetlania pomocy. Opisz działanie i zwracany wynik.

PRZYKŁAD

```
# plik matma.py

def dodaj(a, b):
    """Zwróć sumę dwóch liczb."""
    return a + b

print(dodaj.__doc__)
```

WYNIK

```
Zwróć sumę dwóch liczb.
```

Pomoc w Pythonie:
help(dodaj)

Dokumentacja modułu w terminalu:
python -m pydoc matma

Dopisz docstring do funkcji z wcześniejszego ćwiczenia.

XML: struktura i odczyt

XML opisuje dane za pomocą zagnieżdżonych elementów. Moduł minidom buduje z dokumentu drzewo węzłów, po którym można się poruszać. parseString() odczytuje XML z napisu, a parse() z pliku.

PRZYKŁAD

```
from xml.dom import minidom

xml = """<znajomi>
  <osoba><imie>Ala</imie></osoba>
</znajomi>"""
dokument = minidom.parseString(xml)
print(dokument.documentElement.tagName)
```

WYNIK

```
znajomi
```

documentElement wskazuje element główny.

Do pliku użyj minidom.parse("plik.xml").

Który element jest dzieckiem elementu znajomi?

XML: wyszukiwanie elementów

getElementsByTagName() wyszukuje elementy o podanej nazwie w drzewie dokumentu. Wynik może zawierać wiele węzłów lub być pusty. Tutaj wybieramy pierwszy element imie i odczytujemy jego tekst.

PRZYKŁAD

```
from xml.dom import minidom

xml = "<znajomi><imie>Ala</imie></znajomi>"
dokument = minidom.parseString(xml)
imiona = dokument.getElementsByTagName("imie")
print(imiona[0].firstChild.data)
```

WYNIK

```
Ala
```

firstChild.data odczytuje tekst w tym przykładzie.

W ogólnym XML element może być pusty lub zawierać kilka węzłów tekstowych.

Co zwróci len(imiona)?

Metody HTTP w API

Żądanie HTTP zawiera metodę i adres zasobu, a czasem także dane do przetworzenia. Metoda opisuje zamiar klienta. Serwer odpowiada kodem statusu i opcjonalną treścią, zgodnie z dokumentacją API.

Metoda	Zastosowanie	Przykładowe żądanie
GET	Odczyt zasobu	GET /osoby/7
POST	Przekazanie danych do przetworzenia	POST /osoby
PUT	Utworzenie lub zastąpienie stanu zasobu	PUT /osoby/7
PATCH	Częściowa zmiana zasobu	PATCH /osoby/7
DELETE	Usunięcie powiązania zasobu z adresem	DELETE /osoby/7
HEAD	Metadane odpowiedzi, bez jej treści	HEAD /osoby/7

Którą metodą zwykle pobierzesz listę danych?

Requests: pobranie danych z API

Requests wysyła żądanie HTTP i udostępnia odpowiedź serwera. Pobieramy dane kraju, sprawdzamy status i odczytujemy JSON. Przykład wymaga zainstalowanego pakietu requests oraz połączenia z internetem.

PRZYKŁAD

```
import requests

url = "https://restcountries.com/v3.1/name/Poland"
odp = requests.get(url, timeout=10)
odp.raise_for_status()
kraj = odp.json()[0]
print(kraj["name"]["common"])
```

PRZYKŁADOWY WYNIK

```
Poland
```

timeout ustala limit oczekiwania.
raise_for_status() zgłasza błąd dla 4xx/5xx.
json() odczytuje treść JSON.

Co może się zdarzyć, gdy serwer nie odpowiada?

Wróć do poznanych pojęć z fiszkami

Kilka minut powtórki dziś.

Kolejna próba jutro.

Zapisz adres lub zrób zdjęcie tego slajdu.



naukapythona.pl


Dziękujemy za uwagę